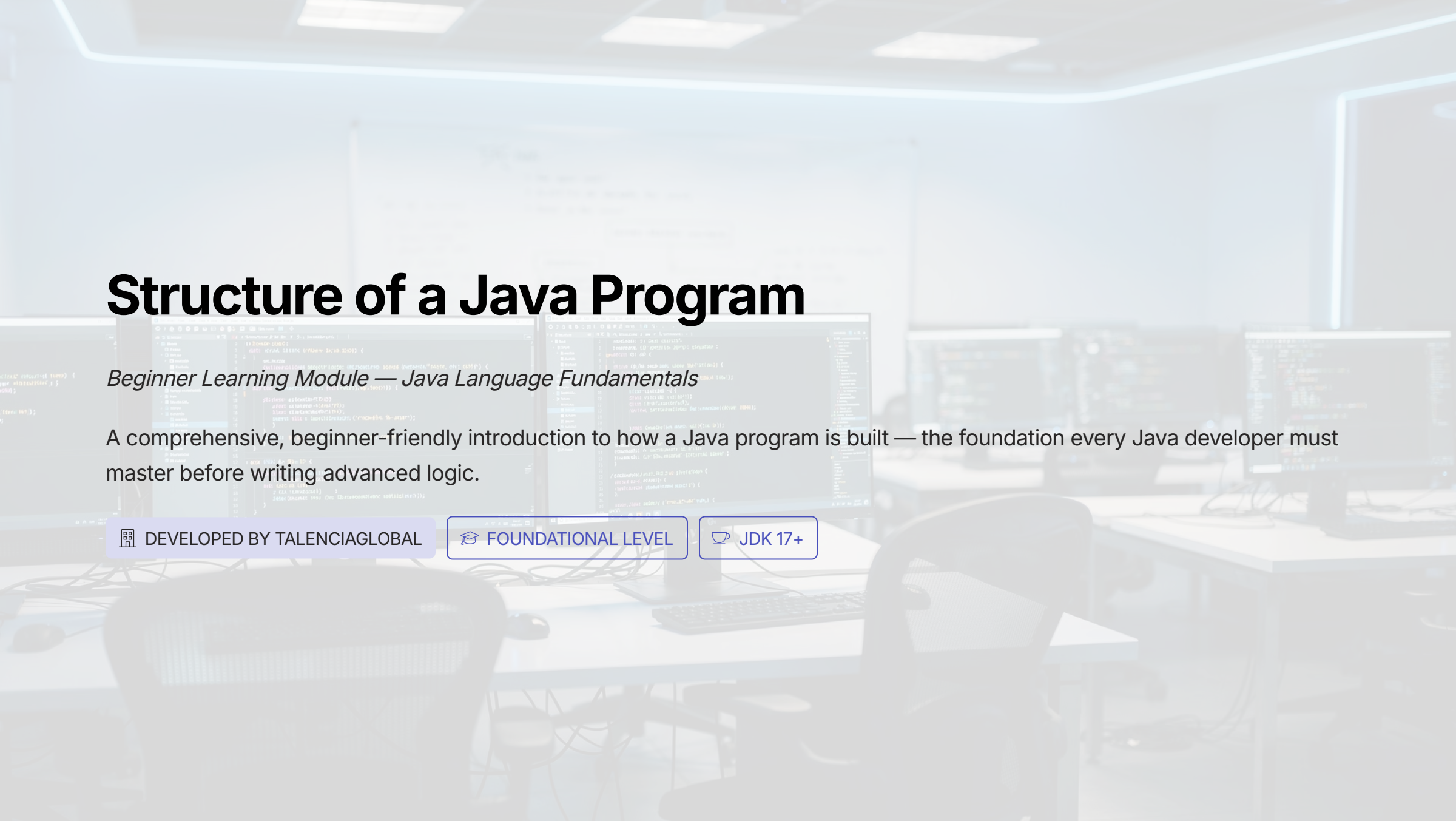


Structure of a Java Program

Beginner Learning Module — Java Language Fundamentals

A comprehensive, beginner-friendly introduction to how a Java program is built — the foundation every Java developer must master before writing advanced logic.

DEVELOPED BY TALENCIAGLOBAL | FOUNDATIONAL LEVEL | JDK 17+

Topic Classification & Industry Context

Java remains one of the most widely deployed programming languages on the planet. Understanding its structural foundations is not optional — it is the prerequisite for every line of professional Java code ever written.

Module Metadata

Topic	Structure of a Java Program
Category	Programming Language Fundamentals
Domain	Java Language Basics
Difficulty	Beginner (Foundational)
Prerequisites	JDK 17+ installed
Relevance	Universal — all Java programs

Recommended Learning Path

01

Structure of a Program

This module — the foundation

02

Variables & Data Types

Storing and typing information

03

Expressions & I/O

Computing and communicating

04

Conditions & Loops

Controlling program flow

ⓘ 📊 **Industry Stat:** As of 2024, Java is used in over **3 billion devices** worldwide and powers more than **90% of Fortune 500 companies'** backend systems. Every one of those programs begins with the exact structure covered in this module.

What Is a Java Program?

Simple Explanation

A **Java program** is a text file (or several) containing instructions written in the Java language. The file carries the extension `.java` and follows a strict structure that the Java compiler enforces without exception.

Unlike Python — where a single line such as `print("Hello")` constitutes a valid program — Java requires every instruction to be wrapped inside a **class**, with a designated **main method** serving as the program's entry point.

Formal Definition

A Java program is a collection of one or more `.java` source files, each containing one or more **classes**. Execution begins from a special method called `main` in a designated class.

> Even to print "Hello, World!" — Java requires the full class and main method wrapper. This is by design, not by accident.

The Smallest Valid Java Program

```
public class Hello {
  public static void main(String[] args) {
    System.out.println("Hello, World!");
  }
}
```

Key Structural Features

File extension	`.java`
Compiled or interpreted?	Compiled to bytecode, run by JVM
Entry point	The `main` method
Wrapping required?	Yes — code lives inside a class
Statement terminator	Semicolon `;`
Code blocks	Curly braces `{ }`
Indentation	Recommended, not enforced

Why Structure Matters — Java's Design Philosophy

Java was engineered for **large-scale enterprise development** where hundreds or thousands of developers contribute to a single codebase. Its strict structural rules are not bureaucratic overhead — they are the mechanism that makes collaborative software engineering possible at scale.

What Java's Structure Provides

Predictability

Every Java program looks similar. Once you can read one, you can read any.

Clear Entry Point

The `main` method tells the JVM — and every developer — exactly where execution begins.

Modularity

Code lives in classes, which can be reused, tested independently, or organized into packages.

Strict Compilation

Many errors are caught at compile time — before the program ever runs in production.

Industry Relevance

35+

Years in Production

Java has been the backbone of enterprise software since 1995

9M+

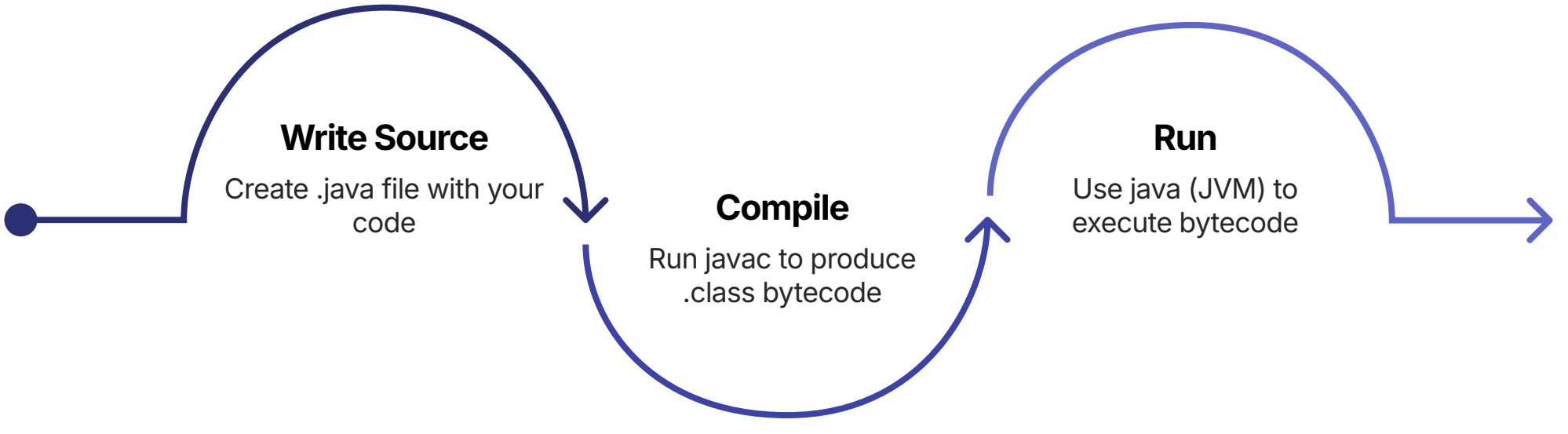
Java Developers

Active Java developers worldwide as of 2024

> *"Write Once, Run Anywhere"* — Java's foundational promise, enabled by compiling source code to bytecode that the JVM executes on any operating system.

How Java Executes Your Program

Java's execution model involves **two distinct steps**: compilation and runtime. This two-phase approach is what enables Java's platform independence — the same bytecode runs on Windows, macOS, Linux, and embedded systems alike.



This compile-then-run cycle is fundamental to understanding Java. The compiler catches structural and type errors before execution; the JVM handles platform-specific execution at runtime.

Two Essential Commands

`javac Hello.java`	Compiles source → creates `Hello.class`
`java Hello`	Runs the compiled bytecode (no `.class` extension)

Three Critical Terms

JDK	Java Development Kit — the full toolkit including `javac`, `java`, and libraries. *You install this.*
JRE	Java Runtime Environment — sufficient to run Java programs
JVM	Java Virtual Machine — the engine that actually executes your `.class` bytecode

The cycle in practice: **JDK gives you**

Anatomy of a Java Program

Every element in a Java source file has a specific role and a specific position. Understanding this anatomy is the difference between writing code that compiles and writing code that communicates intent clearly to both the compiler and your colleagues.

```
// Hello.java
// A simple greeting program

public class Hello {                    // class declaration
    public static void main(String[] args) {  // main method
        System.out.println("Hello, World!");  // statement
    }                                   // end of main
}                                       // end of class
```

The Building Blocks — Reference Table

Block	Purpose	Required?
Package declaration	Groups related classes into namespaces	Optional for beginners
Import statements	Bring in classes from other packages	Only when needed
Class declaration	The wrapper for all your code	✅ Required
main method	The entry point of the program	✅ Required (runnable program)
Statements	Instructions that perform work	✅ Required (or nothing happens)
Braces `{ }`	Group code into logical blocks	✅ Required
Semicolons `;`	Terminate each statement	✅ Required
Comments	Notes for human readers; ignored by compiler	Optional but strongly recommended

The Class Declaration & The main Method

Class Declaration Syntax

```
public class ClassName {
    // your code here
}
```

public

Access modifier — visible everywhere

class

The keyword declaring a class

ClassName

PascalCase by convention

{ }

Opening and closing braces — the class body

⚠ **Golden Rule:** One public class per file, named *exactly* the same as the file (case-sensitive). `Hello.java` → `public class Hello`

The main Method — Exact Signature

```
public static void main(String[] args) {
    // your code here
}
```

`public`	Accessible from outside the class
`static`	Belongs to the class, not an object instance
`void`	Returns nothing
`main`	The method name — must be exactly `main`
`String[] args`	Array of command-line arguments

You do not need to fully understand `public`, `static`, `void`, or `String[] args` right now. **Memorize the line exactly as shown.** Every Java program needs it.

Valid Variations (Use the First)

```
public static void main(String[] args) //
```

Statements, Semicolons, Braces & Indentation

Statements & Semicolons

A **statement** is a complete instruction that Java executes. **Every statement ends with a semicolon** — this is non-negotiable.

```
int age = 25; //
```

Comments, Imports & Package Declarations

Three Types of Comments

```
// Single-line comment

/*
 * Multi-line comment.
 * Spans as many lines as needed.
 */

/**
 * JavaDoc comment — generates documentation.
 * @param length the length of the rectangle
 * @return the area
 */
```

Explain *why*, not *what*

Good comments explain intent, not the obvious mechanics of the code.

Mark TODOs

`// TODO: implement validation` — a universally recognized convention in professional teams.

File-Level Documentation

Add a comment block at the top of every file describing its purpose and author.

Import Statements

Java has thousands of pre-written classes organized into packages. To use one outside your current package, you must import it.

```
// 1. Package (optional)
package com.example.app;

// 2. Imports
import java.util.Scanner;
import java.time.LocalDate;

// 3. Class
public class MyProgram {
    public static void main(String[] args) {
        // ...
    }
}
```

java.lang is auto-imported. That is why `String`, `System`, `Math`, and `Integer` require no import statement — they are always available.

Package Declaration

A **package** groups related classes like folders group files. If declared, it must be the **very first line** of the file. Beginners may safely omit it — Java places the class in the default unnamed package.

```
package com.example.banking; // must match folder path
```

File Naming, Keywords & Identifiers

The Golden File Naming Rule

The file name must **exactly match** the name of the public class inside it — including case — with the extension `.java`.

File	Class Inside	Valid?
`Hello.java`	`public class Hello`	✅
`Hello.java`	`public class hello`	❌ Case mismatch
`Hello.java`	`public class Greeting`	❌ Names differ
`Greeting.java`	`public class Greeting`	✅

Reserved Keywords

Java has **53 reserved keywords** that cannot be used as identifiers. Your IDE will highlight them automatically. Attempting to use one as a variable name produces a compile error.

```
int class = 5; //
```

A Complete, Well-Structured Java Program

The following program demonstrates every structural element covered in this module — applied together in a realistic, professional pattern that scales from beginner exercises to enterprise applications.

```java
// File: ShoppingTotal.java
// Calculates the total bill for a shopping order.

/**
 * A simple program demonstrating Java program structure.
 * Author: Your Name
 */
public class ShoppingTotal {

    public static void main(String[] args) {

        // --- 1. Constants ---
        final double TAX_RATE = 0.18; // 18% tax

        // --- 2. Inputs (hardcoded for now) ---
        String itemName = "Coffee Mug";
        double unitPrice = 250.0;
        int quantity = 3;

        // --- 3. Computation ---
        double subtotal = unitPrice * quantity;
        double tax      = subtotal * TAX_RATE;
        double total    = subtotal + tax;

        // --- 4. Output ---
        System.out.println("=== Order Summary ===");
        System.out.println("Item:     " + itemName);
        System.out.println("Qty:      " + quantity);
        System.out.println("Subtotal: " + subtotal);
        System.out.println("Tax:      " + tax);
        System.out.println("Total:    " + total);
    }
}
```

Program Output

```
=== Order Summary ===
Item:     Coffee Mug
Qty:      3
Subtotal: 750.0
Tax:      135.0
Total:    885.0
```

Structural Checklist

01

File-Level Comments

Describe the file's purpose at the top

02

JavaDoc

Formal documentation for the class

03

Class Declaration

Matches the file name exactly

04

main Method

Exact signature — the entry point

05

Constants → Variables → Compute → Output

A pattern that scales to enterprise applications

Hands-On Labs

Practical application is the fastest path to retention. These two labs are designed to be completed sequentially, reinforcing the write → compile → run cycle and the ability to diagnose structural errors — a skill used daily by professional Java developers.

Lab 1 — Your First Java Program

BEGINNER

Objective: Write, compile, and run your first Java program from scratch.

1

Create the File

Create a folder `java-basics`. Inside it, create a file named exactly `Greeting.java`.

2

Write the Code

Type the class and main method with `System.out.println("Hello, " + name + "!");`

3

Compile

Run `javac Greeting.java` in the terminal. Verify `Greeting.class` appears.

4

Run

Execute `java Greeting`. Expected output: `Hello, World!`

Troubleshooting Guide

Error	Fix
'javac' not recognized	JDK not on PATH — reinstall or set PATH
should be declared in Greeting.java	File name doesn't match class name
';' expected	Missing semicolon at end of statement
'}' expected	Mismatched braces — check every `{`

Lab 2 — Fix the Broken Program

BEGINNER+

Objective: Read structurally flawed Java code and rewrite it correctly. This mirrors real-world code review — a daily activity in professional development teams.

Broken Code (bad.java)

```
public class greeting {
 public static void main(String args[]) {
  String name="Alice"
  int Age = 25
  System.out.println("Hi "+name+
   " you are "+age+" years old")
}}
```

Problems to Fix

1. Class name `greeting` violates PascalCase; file name mismatch
2. Missing semicolons after every statement
3. Inconsistent variable naming: `Age` vs `age`
4. Brace style — convert to K&R Style 1
5. No comments or file-level documentation

Expected Clean Output

```
Hi Alice, you are 25 years old.
```

Learning Outcome: Recognizing and correcting structural mistakes is a skill exercised in every professional code review. Studies show developers spend up to **20% of their working time** reviewing and correcting code written by others.

Java vs Python — Structural Comparison

Understanding Java's structure is sharpened by contrast. The comparison below illustrates why Java's verbosity is a deliberate architectural choice, not a deficiency — and why it dominates enterprise environments where Python's brevity would introduce ambiguity at scale.

Java — Hello, World!

```java
public class Hello {
    public static void main(String[] args) {
        System.out.println("Hello, World!");
    }
}
```

Python — Hello, World!

```python
print("Hello, World!")
```

That is a **massive** structural difference — 5 lines vs 1. Both are correct in their respective languages.

Aspect	Java	Python
File extension	`.java`	`.py`
Class required?	✅ Yes — every program	❌ No
main method required?	✅ Yes	❌ No
Compilation step?	✅ `javac`	❌ Interpreted directly
Statement terminator	Semicolon `;`	Newline
Code blocks	Curly braces `{}`	Indentation
Indentation enforced?	❌ Recommended only	✅ Required by language
File name = class name?	✅ Required	❌ Free
Type declarations	✅ Mandatory	❌ Inferred
Boilerplate level	High	Minimal

When Python Wins

Quick scripts, data science prototypes, automation tasks, and rapid iteration where minimal boilerplate accelerates delivery.

When Java Wins

Large enterprise applications, Android development, financial systems, and any environment where strict structure enables thousands of developers to collaborate reliably.

Best Practices & Common Pitfalls

✅ Professional Do's

1 One public class per file

File name must match the class name exactly, including case.

2 Indent with 4 spaces

Configure your IDE's Tab key to insert 4 spaces per level.

3 End every statement with ;

Non-negotiable. No exceptions.

4 Use K&R brace style

Opening brace on the same line — the Java community standard.

5 Follow naming conventions

PascalCase for classes, camelCase for variables, UPPER_SNAKE for constants.

6 Run the IDE formatter

Use Ctrl+Alt+L (IntelliJ) or equivalent before every commit.

❌ Critical Don'ts

Don't forget the semicolon

The #1 beginner error — responsible for the majority of first-time compile failures.

Don't mismatch braces

Every `{` needs a `}`. Use your IDE's brace-matching highlight.

Don't use keywords as identifiers

`class`, `int`, `if`, `for` — all reserved. Your IDE will flag them.

Don't put main() outside a class

It must be inside the class body — always.

Don't use .class in the java command

`java Hello` ✅ — `java Hello.class` ❌

Don't forget to recompile after edits

Always run `javac` again after modifying source — otherwise you run stale bytecode.

⊗ **Top 3 Compile Errors by Frequency (Introductory Java Courses):** (1) Missing semicolon — ~40% of errors; (2) Mismatched braces — ~25%; (3) File/class name mismatch — ~15%. Mastering these three rules eliminates **80% of beginner compile failures**.

Learning Summary & Revision Cheat Sheet

The following reference consolidates every critical rule from this module into a single, scannable resource. Return to this card whenever you need a quick structural reminder.

10 Key Takeaways

01

Every Java program is a .java file

Containing one or more classes.

02

The main method is the entry point

`public static void main(String[] args)` — memorize this exactly.

03

Compile with javac, run with java

Two commands, two distinct purposes.

04

Every statement ends with ;

No exceptions.

05

Braces { } group code into blocks

Every opening brace must have a matching closing brace.

06

File name must match the public class name

Case-sensitive. Always.

07

Java is case-sensitive

Reserved keywords cannot be used as identifiers.

08

K&R brace style, 4-space indentation

The professional Java community standard.

09

java.lang is auto-imported

Everything else requires an explicit import statement.

10

Comments use //, /* */, or /** */

JavaDoc comments generate formal API documentation.

Quick Reference Cheat Sheet

```
FILE EXTENSION: .java
COMPILE: javac Hello.java
RUN: java Hello (no .class!)

MINIMUM PROGRAM
public class Hello {
 public static void main(String[] args) {
 System.out.println("Hi");
 }
}

FILE
```